Tema 1: Genéricos

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Introducción

- Si queremos implementar una clase que maneje un array de enteros, podemos escribir:

```
public class IntList {
  private final static int _INIT_SIZE = 2;
  private Integer[] elem;
  private int last;

  public IntList() {
    last = -1;
    elem = new Integer[_INIT_SIZE];
  }

  public void addElem(Integer e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public Integer getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

Introducción

- Si queremos implementar una clase que maneje un array de enteros, podemos escribir:

```
public class IntList {
  private final static int _INIT_SIZE = 2;
  private Integer[] elem;
  private int last;

  public IntList() {
    last = -1;
    elem = new Integer[_INIT_SIZE];
  }

  public void addElem(Integer e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public Integer getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
public static void main(String[] args) {
  IntList x = new IntList();
  x.addElem(12);
  x.addElem(244);
  x.addElem(31);
  System.out.println(x.getElem(0));
  System.out.println(x.getElem(1));
  System.out.println(x.getElem(2));
}
```

Introducción

- Qué pasa si ahora queremos que sea una lista de objetos de tipo **String**

```
public class StringList {
  private final static int _INIT_SIZE = 2;
  private String[] elem;
  private int last;

  public StringList() {
    last = -1;
    elem = new String[_INIT_SIZE];
  }

  public void addElem(String e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public String getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

Introducción

- Qué pasa si ahora queremos que sea una lista de objetos de tipo **String**

```
public class StringList {
  private final static int _INIT_SIZE = 2;
  private String[] elem;
  private int last;

  public StringList() {
    last = -1;
    elem = new String[_INIT_SIZE];
  }

  public void addElem(String e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public String getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
public static void main(String[] args) {
  StringList x = new StringList();
  x.addElem("Hola");
  x.addElem("Hey");
  x.addElem("Bla");
  System.out.println(x.getElem(0));
  System.out.println(x.getElem(1));
  System.out.println(x.getElem(2));
}
```

Mantener varias Implementaciones

- IntList y StringList son iguales salvo el tipo.
- Mantener dos implementaciones complica, por ejemplo, el mantenimiento de código.
- ¿Qué pasa si encontramos un error en la implementación?
- ¿Y si queremos una lista de tipo Double o de tipo File?
- Tenemos que pensar cómo tener una sola implementación que se pueda usar con cualquier tipo. .

Mantener varias Implementaciones

- IntList y StringList son iguales salvo el tipo.
- Mantener dos implementaciones complica, por ejemplo, el mantenimiento de código.
- ¿Qué pasa si encontramos un error en la implementación?
- ¿Y si queremos una lista de tipo Double o de tipo File?
- Tenemos que pensar cómo tener una sola implementación que se pueda usar con cualquier tipo. **Usando genéricos**.

Generalización. Primera Aproximación

- Si en lugar de un tipo específico del programa utilizamos **Object**, sirve para cualquier tipo no primitivo. Cualquier tipo es un subtipo de Object.
- Para los tipos primitivos podemos utilizar los tipos no primitivos equivalentes (Integer, etc.) y boxing/unboxing.

```
public class ObjList {
  private final static int _INIT_SIZE = 2;
  private Object[] elem;
  private int last;

  public ObjList() {
    last = -1;
    elem = new Object[_INIT_SIZE];
  }

  public void addElem(Object e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public Object getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

Generalización. Primera Aproximación

- Si en lugar de un tipo específico del programa utilizamos **Object**, sirve para cualquier tipo no primitivo. Cualquier tipo es un subtipo de Object.
- Para los tipos primitivos podemos utilizar los tipos no primitivos equivalentes (Integer, etc.) y boxing/unboxing.

```
public class ObjList {
  private final static int _INIT_SIZE = 2;
  private Object[] elem;
  private int last;

  public ObjList() {
    last = -1;
    elem = new Object[_INIT_SIZE];
  }

  public void addElem(Object e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public Object getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
public static void main(String[] args) {
  ObjList x = new ObjList();
  x.addElem("Hola");
  x.addElem("Hey");

  ObjList y = new ObjList();
  y.addElem(1);
  y.addElem(33);
}
```

Generalización. Primera Aproximación

- Si en lugar de un tipo específico del programa utilizamos **Object**, sirve para cualquier tipo no primitivo. Cualquier tipo es un subtipo de Object.
- Para los tipos primitivos podemos utilizar los tipos no primitivos equivalentes (Integer, etc.) y boxing/unboxing.

```
public class ObjList {
  private final static int _INIT_SIZE = 2;
  private Object[] elem;
  private int last;

  public ObjList() {
    last = -1;
    elem = new Object[_INIT_SIZE];
  }

  public void addElem(Object e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public Object getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
public static void main(String[] args) {
  ObjList x = new ObjList();
  x.addElem("Hola");
  x.addElem("Hey");

  ObjList y = new ObjList();
  y.addElem(1);
  y.addElem(33);
}
```

¿ Cuál es el problema de esta idea?

Es la Manera Correcta de Hacerlo?

En principio no hay ningún problema. Es la "mejor" manera de generalizar las listas. Pero usar esta clase puede resultar difícil o generar inconvenientes:

- Es necesario hacer **cast** cuando se extraen elementos para que el programa compile. Necesitamos el tipo esperado!

```
ObjList x = new ObjList();
x.addElem(100);
...
Integer y = x.getElem(0);
System.out.println(2*y);
```

Es la Manera Correcta de Hacerlo?

En principio no hay ningún problema. Es la "mejor" manera de generalizar las listas. Pero usar esta clase puede resultar difícil o generar inconvenientes:

- Es necesario hacer **cast** cuando se extraen elementos para que el programa compile. Necesitamos el tipo esperado!

```
ObjList x = new ObjList();
x.addElem(100);
...
Integer y = x.getElem(0);
System.out.println(2*y);
```

No compila

Es la Manera Correcta de Hacerlo?

En principio no hay ningún problema. Es la "mejor" manera de generalizar las listas. Pero usar esta clase puede resultar difícil o generar inconvenientes:

- Es necesario hacer **cast** cuando se extraen elementos para que el programa compile. Necesitamos el tipo esperado!

```
ObjList x = new ObjList();
x.addElem(100);
...
Integer y = x.getElem(0);
System.out.println(2*y);
```

No compila

En Java, una variable de tipo T solo puede tener un valor de un subtipo de T.

T1 es un subtipo de T2 si T1 = T2 o T1 implementa/extiende T2

Es la Manera Correcta de Hacerlo?

En principio no hay ningún problema. Es la "mejor" manera de generalizar las listas. Pero usar esta clase puede resultar difícil o generar inconvenientes:

- Es necesario hacer **cast** cuando se extraen elementos para que el programa compile. Necesitamos el tipo esperado!

```
ObjList x = new ObjList();
x.addElem(100);
...
Integer y = x.getElem(0);
System.out.println(2*y);
```

No compila

x.getElem(0) devuelve **Object**, mientras que y es de tipo **Integer**. Sin emabrgo Object no es un subtipo de Integer!

Es la Manera Correcta de Hacerlo?

En principio no hay ningún problema. Es la "mejor" manera de generalizar las listas. Pero usar esta clase puede resultar difíficil o generar inconvenientes:

- Es necesario hacer **cast** cuando se extraen elementos para que el programa compile. Necesitamos el tipo esperado!

```
ObjList x = new ObjList();
x.addElem(100);
...
Integer y = (Integer) x.getElem(0);
System.out.println(2*y);
```

```
ObjList x = new ObjList();
x.addElem(``Hola!'');
...
String y = (String) x.getElem(0);
System.out.println(y);
```

Es la Manera Correcta de Hacerlo?

En principio no hay ningún problema. Es la "mejor" manera de generalizar las listas. Pero usar esta clase puede resultar difíficil o generar inconvenientes:

- Es necesario hacer **cast** cuando se extraen elementos para que el programa compile. Necesitamos el tipo esperado!
- Pero este código es **inseguro**. ¿Qué pasa si hacemos un casting para convertir a un tipo incorrecto?

```
ObjList x = new ObjList();
x.addElem(100);
...
String y = (String) x.getElem(0);
System.out.println(y);
```

Es la Manera Correcta de Hacerlo?

En principio no hay ningún problema. Es la "mejor" manera de generalizar las listas. Pero usar esta clase puede resultar difícil o generar inconvenientes:

- Es necesario hacer **cast** cuando se extraen elementos para que el programa compile. Necesitamos el tipo esperado!
- Pero este código es **inseguro**. ¿Qué pasa si hacemos un casting para convertir a un tipo incorrecto?

```
ObjList x = new ObjList();
x.addElem(100);
...
String y = (String) x.getElem(0);
System.out.println(y);
```

Tendremos un error de ejecución (runtime error) porque los tipos no coinciden — ClassCastException

Comprobación de tipos

- **Estático**: el compilador detecta que los tipos no coinciden. El programa no termina.

```
Integer x = new Integer(1);
Double y = x;
```

- **Dinámico**: el problema de tipos se detecta durante la ejecución. El programa compila pero Java lanza una excepción.

```
Object x = new Integer(1);
Double y = (Double) x;
```

La comprobación dinámica es útil para el desarrollo rápido, pero la estática es mejor para sistemas grandes, ya que nos prohibe hacer cosas "peligrosas".

Solución: Genéricos en Java

- Es una utilidad de Java, introducida en J2SE 5.0 en 2005, que permite definir clases e interfaces sobre un **tipo genérico**, evitando los problemas anteriores.
- Los genéricos de Java son *azúcar sintáctico* para no tener que hacer *cast* inseguros continuamente.
- La idea de los genéricos consiste en **parametrizar** la definición de una clase respecto a un tipo genérico.
- Para ello vamos a utilizar *parámetros* para representar un tipo de datos (también llamados *variables de tipo*).
- **Cuando se crea un objeto de esta clase** en otro punto del programa, se especifica el tipo concreto que va a tomar el parámetro.
- **Notación**: Es común representar los parámetros con identificadores formados por una sola letra mayúscula.

Solución: Genéricos en Java

- La cabecera de una clase genérica tiene, por normal general, la siguiente forma:
  public class ClaseGenerica<T extends OtraClase>
- Si **T** extiende de la clase **Object**, se puede omitir la herencia.
  public class ClaseGenerica<T extends Object>
  public class ClaseGenerica<T>
- **OtraClase** actúa como cota superior y hace que solo se pueda instanciar con clases que sean subclases de **OtraClase**.

ObjList usando Genéricos

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = (T[]) Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

ObjList usando Genéricos

T es un parámetro de tipo.

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = (T[]) Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

ObjList usando Genéricos

T es un parámetro de tipo.

Object es la cota superior de **T**.

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = (T[]) Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

ObjList usando Genéricos

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = (T[]) Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

T es un parámetro de tipo.

Object es la cota superior de **T**.

Se puede usar MyList<X> con cualquier X, mientras sea un subtipo de la cota superior de **T**.

`MyList<Integer>`

`MyList<String>`

ObjList usando Genéricos

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = (T[]) Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

T es un parámetro de tipo.

Object es la cota superior de **T**.

Se puede usar MyList<X> con cualquier X, mientras sea un subtipo de la cota superior de **T**.

`MyList<Integer>`

`MyList<String>`

`T extends Number` permite instanciar T con cualquier subtipo de Number como `MyList<Integer>` o `MyList<Double>` pero no `MyList<String>`

ObjList usando Genéricos

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = ( T[]) new Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

CUIDADO! No se pueden crear instancias o arrays de **T**. Normalmente hay que usar la cota superior.

Cómo usar la Lista Genérica

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = (T[]) Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

Cómo usar la Lista Genérica

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = (T[]) Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem = Arrays.copyOf(elem, elem.length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
MyList<String> x = new MyList<String>();
x.addElem(100); \\ERROR
x.addElem("Hola"); \\OK
...
String y = x.getElem(0); \\OK
Integer y = x.getElem(0); \\ERROR
```

```
MyList<Integer> x = new MyList<Integer>();
x.addElem(100); \\OK
x.addElem("Hola"); \\ERROR
...
String y = x.getElem(0); \\ERROR
Integer y = x.getElem(0); \\OK
```

Cómo Compila Java los Genéricos

- Los programas Java se compilan a Java Bytecode. Sin embargo, en Java Bytecode no hay genéricos. ¿Cómo se compilan entonces?
- Para compilar un genérico, **Java reemplaza los parámetros por sus cotas superiores** (*Type-Erasures*).
- El compilador usa inferencia de tipos para detectar posibles errores y añade casting cuando es necesario.

Cómo Compila Java los Genéricos

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = ( T[]) new Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem =
          Arrays.copyOf(elem, elem.
         length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
public class MyList {
  private final static int _INIT_SIZE = 2;
  private Object[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = new Object[_INIT_SIZE];
  }

  public void addElem(Object e) {
    if (last == elem.length - 1) elem
          = Arrays.copyOf(elem, elem.
         length * 2);
    elem[++last] = e;
  }

  public Object getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

Cómo Compila Java los Genéricos

```
public class MyList<T extends Object> {
  private final static int _INIT_SIZE = 2;
  private T[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = ( T[]) new Object[_INIT_SIZE];
  }

  public void addElem(T e) {
    if (last == elem.length - 1) elem =
          Arrays.copyOf(elem, elem.
         length * 2);
    elem[++last] = e;
  }

  public T getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
public class MyList {
  private final static int _INIT_SIZE = 2;
  private Object[] elem;
  private int last;

  public MyList() {
    last = -1;
    elem = new Object[_INIT_SIZE];
  }

  public void addElem(Object e) {
    if (last == elem.length - 1) elem
          = Arrays.copyOf(elem, elem.
         length * 2);
    elem[++last] = e;
  }

  public Object getElem(int index) {
    if (index > last) return null;
    else return elem[index]
  }
}
```

```
MyList<Integer> x = new MyList<Integer>();
x.addElem(100);
Integer y = x.getElem(0);
```

```
MyList x = new MyList();
x.addElem(100);
Integer y = (Integer) x.getElem(0);
```

Por qué Tengo que Usar Genéricos

Si el compilador reemplaza los parámetros de tipo con las cotas superiores, al compilar MyList<T> obtenemos ObjList. Entonces, ¿cuál es la ventaja de usar genéricos?

Por qué Tengo que Usar Genéricos

Si el compilador reemplaza los parámetros de tipo con las cotas superiores, al compilar MyList<T> obtenemos ObjList. Entonces, ¿cuál es la ventaja de usar genéricos?

- Comprobación estática de tipos: Es más seguro. El compilador usa inferencia de tipos para detectar errores en MyList<T>. Esto no lo puede hacer con ObjList porque no sabe cuál es el parámetro de tipo.

Por qué Tengo que Usar Genéricos

Si el compilador reemplaza los parámetros de tipo con las cotas superiores, al compilar MyList<T> obtenemos ObjList. Entonces, ¿cuál es la ventaja de usar genéricos?

- Comprobación estática de tipos: Es más seguro. El compilador usa inferencia de tipos para detectar errores en MyList<T>. Esto no lo puede hacer con ObjList porque no sabe cuál es el parámetro de tipo.
- Nos permite escribir algoritmos genéricos: Podemos implementar algoritmos de forma genérica sin tener que repetir código.

Por qué Tengo que Usar Genéricos

Si el compilador reemplaza los parámetros de tipo con las cotas superiores, al compilar MyList<T> obtenemos ObjList. Entonces, ¿cuál es la ventaja de usar genéricos?

- Comprobación estática de tipos: Es más seguro. El compilador usa inferencia de tipos para detectar errores en MyList<T>. Esto no lo puede hacer con ObjList porque no sabe cuál es el parámetro de tipo.
- Nos permite escribir algoritmos genéricos: Podemos implementar algoritmos de forma genérica sin tener que repetir código.
- Eliminamos el casting: Eliminamos una fuente de error. El casting lo hace el compilador.

Múltiples Parámetros de Tipo

Una clase puede tener varios parámetros de tipo

```
public class Pair<A,B> {
  A first;
  B second;

  public Pair(A f, B s) {
    this.first = f;
    this.second = s;
  }

  public A getFirst() {
    return first;
  }

  public B getSecond() {
    return second;
  }
}
```

Múltiples Parámetros de Tipo

Una clase puede tener varios parámetros de tipo

```
public class Pair<A,B> {
  A first;
  B second;

  public Pair(A f, B s) {
    this.first = f;
    this.second = s;
  }

  public A getFirst() {
    return first;
  }

  public B getSecond() {
    return second;
  }
}
```

```
Pair<A extends Number, B extends Number>
Pair<A, B extends Number>
Pair<A extends Bla, B extends Number>
Pair<A extends Number, B extends MyList<A>>
...
```

Interfaces Genéricas

```
public interface ListI<T> {
  public void addElem(T e);
  public T getElem(int index);
  public int size();
}
```

Interfaces Genéricas

```
public interface ListI<T> {
  public void addElem(T e);
  public T getElem(int index);
  public int size();
}
```

```
public class MyList<T> implements ListI<T> {
  ...
  @Overrride
  public void addElem(T e) {...}

  @Overrride
  public T getElem(int index) {...}

  @Overrride
  public int size() {...}
}
```

Interfaces Genéricas

```
public interface ListI<T> {
  public void addElem(T e);
  public T getElem(int index);
  public int size();
}
```

```
public class MyList<T> implements
      ListI<T> {
      ...
      @Overrride
      public void addElem(

      @Overrride
      public T getElem(int

      @Overrride
      public int size() {.
      }
```

```
public class MyList implements ListI<Integer> {
  ...
  @Overrride
  public void addElem(Integer e) {...}

  @Overrride
  public Integer getElem(int index) {...}

  @Overrride
  public int size() {...}
}
```

Extender Clases Genéricas

```
public abstract class ListI<T> {
  public abstract void addElem(T e);
  public abstract T getElem(int index);
  public abstract int size();
}
```

Extender Clases Genéricas

```
public abstract class ListI<T> {
  public abstract void addElem(T e);
  public abstract T getElem(int index);
  public abstract int size();
}
```

```
public class MyList<T> extends ListI<T> {
  ...
  ...

  @Overrride
  public void addElem(T e) {...}

  @Overrride
  public T getElem(int index) {...}

  @Overrride
  public int size() {...}
}
```

Métodos Genéricos

- Igual que se pueden definir clases genéricas, se pueden definir **métodos genéricos**.
- Son métodos con parámetros de tipo (no de una clase genérica).
- La clase no es genérica, solo el método.

Métodos Genéricos

- Igual que se pueden definir clases genéricas, se pueden definir **métodos genéricos**.
- Son métodos con parámetros de tipo (no de una clase genérica).
- La clase no es genérica, solo el método.

```
public static <A,B extends Number> boolean compare(Pair<A,B> p1, Pair<A,B>p2){
  return p1.getFirst().equals(p2.getFirst()) &&
     p2.getSecond().equals(p2.getSecond());
}
```

1. Declaración de los parámetros.
2. Uso de los parámetros.

Métodos Genéricos

- Igual que se pueden definir clases genéricas, se pueden definir **métodos genéricos**.
- Son métodos con parámetros de tipo (no de una clase genérica).
- La clase no es genérica, solo el método.

```
public static <A,B extends Number> boolean compare(Pair<A,B> p1, Pair<A,B>p2){
  return p1.getFirst().equals(p2.getFirst()) &&
     p2.getSecond().equals(p2.getSecond());
}
```

1. Declaración de los parámetros.
2. Uso de los parámetros.

```
Pair<Number,String> x = new Pair<Number,String>(1,"Bla!");
Pair<Number,String> y = new Pair<Number,String>(2,"Bla!");
boolean z = <Number,String>compare(x,y);
```

Métodos Genéricos

```
public static <T> void print(MyList<T> x) {
  for(int i=0; i<x.size(); i++) {
    System.out.println(x);
  }
}
```

```
MyList<Integer> x = new MyList<Integer>();
x.addElem(1);
x.addElem(2);
print(x);
MyList<String> y = new MyList<String>();
y.addElem("Bla!");
y.addElem("Hola!");
print(y);
```

Genéricos y Subtipos

¿Es correcto este código?¿Compila?¿Lanza alguna excepción?

```
MyList<Integer> x = new MyList<Integer>();
...
MyList<Number> y = x;
```

Genéricos y Subtipos

¿Es correcto este código?¿Compila?¿Lanza alguna excepción?

```
MyList<Integer> x = new MyList<Integer>();
...
MyList<Number> y = x;
```

Type mismatch!

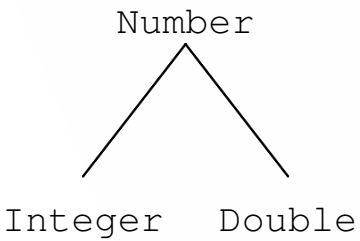
Genéricos y Subtipos

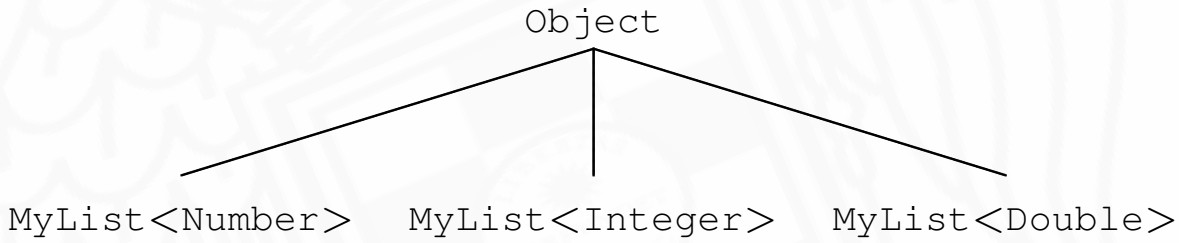
¿Es correcto este código?¿Compila?¿Lanza alguna excepción?

```
MyList<Integer> x = new MyList<Integer>();
...
MyList<Number> y = x;
```

Type mismatch!

El tipo de x tiene que ser subtipo del tipo de y, pero MyList<Integer> no es subtipo de MyList<Number>, aunque Integer es un subtipo de Number!!





Genéricos y Subtipos

¿Es correcto este código?¿Compila?¿Lanza alguna excepción?

```
public static void print(MyList<Number> x) {
  for(int i=0; i<x.size(); i++) {
    System.out.println(x);
  }
}
```

```
MyList<Integer> x = new MyList<Integer>();
...
print(x);
```

Genéricos y Subtipos

¿Es correcto este código?¿Compila?¿Lanza alguna excepción?

```
public static void print(MyList<Number> x) {
  for(int i=0; i<x.size(); i++) {
    System.out.println(x);
  }
}
```

```
MyList<Integer> x = new MyList<Integer>();
...
print(x);
```

Type mismatch! El mismo problema de antes

Genéricos y Subtipos

¿Cómo podemos arreglar el problema anterior? Este código **SI** que funciona:

```
public static <T extends Number> void print(MyList<T> x) {
  for(int i=0; i<x.size(); i++) {
    System.out.println(x);
  }
}
```

```
MyList<Integer> x = new MyList<Integer>();
...
print(x);
```

Pero no podemos ejecutar x.addElem(new Integer()) dentro de print. Porque x.addElem espera algún subtipo de Number pero no sabemos cuál es ...

Wildcards

- El signo **?**, cuando se utiliza en un genérico, indica que es un tipo desconocido. Se conoce en inglés como **wildcards**.

Wildcards

- El signo **?**, cuando se utiliza en un genérico, indica que es un tipo desconocido. Se conoce en inglés como **wildcards**.
- En muchas ocasiones utilizar una variable de tipo **T** o un *wildcard* **?** es lo mismo. Por ejemplo es igual usar **List**<**T**> que usar **List**<**?**>, ya que en ambos casos significa que la lista se puede instanciar con cualquier tipo (que extienda a **Object**).

Wildcards

- El signo **?**, cuando se utiliza en un genérico, indica que es un tipo desconocido. Se conoce en inglés como **wildcards**.
- En muchas ocasiones utilizar una variable de tipo **T** o un *wildcard* **?** es lo mismo. Por ejemplo es igual usar **List**<**T**> que usar **List**<**?**>, ya que en ambos casos significa que la lista se puede instanciar con cualquier tipo (que extienda a **Object**).
- Sin embargo existen diferencias de uso que son importantes:
  1. EL uso de **?** es menos restrictivo que usar un parámetro de tipo. No es lo mismo escribir <**?**,**?**> que <**T**,**T**>. El segundo obliga a que ambos tipos sean el mismo.
  2. **?** admite ser usado con cotas superiores e inferiores. Los parámetros de tipo solo con cotas superiores.

Wildcards

- **Upper Bounded Wildcards**

```
public static void print(MyList<? extends Number> list) {...}
```

- **UnBounded Wildcards**

```
public static void print(MyList<?> list) {
  for (Object el: list)
    System.out.print(el+" ");
}
```

- **Lower Bounded Wildcards**

```
public static void addNumbers(MyList<? super Integer> list) {
  for (int i = 1; i<=10;i++)
    list.add(i);
}
```

Restricciones de los Tipos Genéricos

- No se pueden instanciar con tipos primitivos:

```
Pair<int,char> p = new Pair<int,char>(9,'a');
```

No compila! Pero se puede hacer usando boxing/unboxing:

```
Pair<Integer,Character> p = new Pair<Integer,Character>(9,'a');
```

- No se pueden hacer instancias de parámetros de tipo. Por ejemplo el siguiente código no compila:

```
public static <T> void append(List<T> list) {
  T el = new T();
  list.add(el);
}
```

Se puede hacer usando *reflection*.

Para más restricciones consultar:

`https://docs.oracle.com/javase/tutorial/java/generics/restrictions.html`

Ejemplos

- Comparable
- Rational
- Stack